

Systems Requirements Documentation - Library Management System Final

Logan Young

- Page 2: Library Management System Proposal
- Page 4: Problem Statements and System Requirements Document
- Page 9: Functional Requirements Specification
- Page 12: Sequence and Activity Diagrams
- Page 20: User Interface Specification
- Page 34: Traceability Matrix and Updates
- Page 37: Server Architecture and System Design
- Page 40: User Interface Design and Implementation
- Page 42: Project Plan
- Page 43: Data Model Implementation
- Page 43: Coding Standards and UI Design [Updated]
- Page 44: Testing Strategy
- Page 45: Error Handling and Debugging
- Page 46: Collaboration and Code Integration
- Page 47: Performance Optimization Plan
- Page 47: Maintenance and Support Strategy
- Page 48: Midterm System Implementation Plan

Library Management System Proposal

Problem Statement

A library management system is used to manage the books in a library through the use of an automated system. Basically, the system tracks the status of books and other media throughout the checkout process (whether a book is checked out or available, in storage, reserved, etc.). It can be difficult to keep track of everything a larger-sized library might carry through typical analog filing systems, so automated management systems are used to keep everything organized.

System Objectives

The library management system (LMS) will contain the entire inventory of the library in a database, converting various properties of books and other media to attributes. It will also keep track of if a book is available or has been loaned out, if it's reserved, if it's damaged, etc. Its primary use is for managing the library's inventory, but there will also be a frontend to allow library members to check out books.

System Requirements

- Let library members check out books.
- Let library members reserve books.
- Let library members request a book from a library in the same network.
- Let library staff mark a book as returned.
- Let library staff create a new entry for a new book the library has received.
- Let library staff delete an entry for a book the library no longer possesses.
- Let library staff mark a user as a new library member.
- Let library staff adjust a book's properties if there's been a mistake in filing (e.g. wrong publish date, wrong author).
- Let library administrator mark a user as new library staff.
- Let library administrator perform various administrative tasks (not too sure what this might entail).

Typical Customers

- Library members (the people who check out the books)
- Library staff (the people who work at the library)
- Library administrator (the person in charge of the library)
- IT Staff (the people who work on the LMS, supposing they're separate from staff)

Project Planning and Development Approach

1. Software
 - a. Frontend: React (or JavaScript if I can't learn React in time)
 - b. Backend: Java
 - c. Database: MySQL
2. Hardware
 - a. PC (or device with an internet browser to access the web app)
 - b. Small server in the library for the database.
3. Network
 - a. [In all honesty, the LMS database and web app shouldn't be so intensive as to need high-speed networking, but it's good to future-proof.]
 - b. Fiber-optic internet with ethernet integration to allow quick uploading and downloading from the server to the web app.

Development Plan

- Weeks 1-2
 - o Conceptualize and complete the system proposal.
- Weeks 3-4
 - o Determine connection between frontend, backend, and database and start construction.
- Weeks 5-7
 - o Build database to organize books and their attributes, build frontend webpage for users to interact with, build backend to connect the two.
- Week 8
 - o Test system with sample entries and actions and record demo for midterm.
- Weeks 9-11
 - o Improve system with feedback from professor. Implement suggested features and bug fixes.
- Weeks 12-14
 - o Write test cases and optionally add some cool last-minute features.
- Week 15

- Test final system and record demo for final.

Problem Statements and System Requirements Document

Logan Young

Customer Problem Statement

Problem Statement

A **Library Management System** keeps track of all the goings-on in a library. It catalogs the entire inventory of the library, keeping track of each attribute of each piece of media. It also logs the library's members, its staff, and its administrator as users. If a member checks out a book the LMS will alter that book's checkout status to reflect that change. A member can also view what books are available and submit requests to hold a book. Other users have more privileges related to the system, such as being able to mark a book as unavailable manually or remove a member's records.

Glossary of Terms

LMS – (Library Management System) A tool to electronically manage a library.

User – An entity that uses the LMS to interact with the library.

Member – An LMS user that can only check out and return books.

Staff – An LMS user with elevated privileges related to library upkeep.

Administrator – LMS staff with further elevated privileges.

Checked-out – Status of a book after a user has reserved it. Unavailable for use by other users until returned.

Hold – Status of a book after a user has submitted a hold request. No user other than the one that requested the hold can check it out after it becomes available.

Attribute – Some quality of a book, such as its title or author.

Privilege – Specific groups of users can access more of the LMS than others.

 Welcome, John Library. LMS	
Library Member Return by 11/20	Books Unavailable On Hold
Copyright, Trademarked, Reserved, Monetized - Logan Young	

System Requirements

Functional Requirements

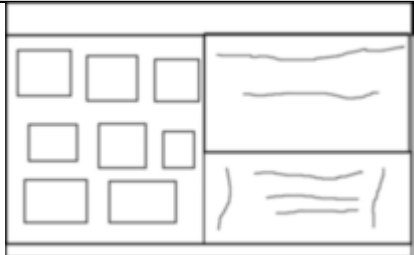
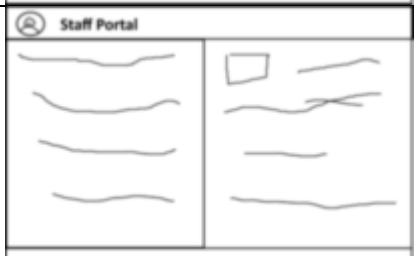
No.	Priority Weight	Description
REQ-1	High	The LMS should be able to store and display books.
REQ-2	High	Library members should be able to use the LMS to check out books.
REQ-3	Medium	Library members should be able to use the LMS to submit a hold request for a maximum of three books.
REQ-4	High	Library staff should be able to use the LMS to mark a book as unavailable.
REQ-5	High	Library members should be able to delete their account from the LMS.
REQ-6	Medium	Library staff should be able to delete a member's account using the LMS.
REQ-7	High	Library staff should be able to convert a new user into a member.
REQ-8	Medium	Library staff should be able to adjust a book's attributes if incorrect.
REQ-9	High	Library staff should be able to create a new entry in the LMS for a newly received book.
REQ-10	Low	Library members should be able to request a book from a different library in the same network.
REQ-11	Medium	Library administrator should be able to convert a new user into staff.
REQ-12	Low	Library administrator should have (nearly) complete access to the LMS.
REQ-13	Medium	Library members should be able to submit a suggestion for new books for the library.

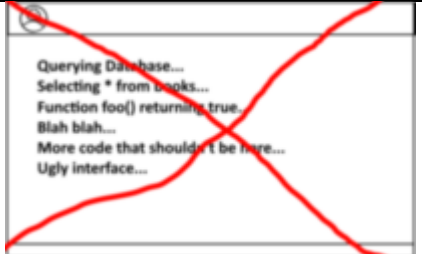
REQ-14	Medium	Library members should be able to create a new donated book entry with approval from staff.
REQ-15	High	LMS users should be able to easily search for a specific book using various filters and sorting techniques.

Nonfunctional Requirements

No.	Priority Weight	Description
N-REQ-1	High	The LMS should be scalable to include as many books as possible.
N-REQ-2	Medium	The LMS should be able to support at least 100 concurrent users.
N-REQ-3	High	The LMS front-end (HTML and CSS) should pass W3C validation with no errors.
N-REQ-4	High	The LMS front-end should pass all test cases for Google Chrome, Microsoft Edge and Mozilla Firefox.
N-REQ-5	Medium	All database processes should be completed within a maximum of 30 seconds.
N-REQ-6	High	LMS users should have password-protected accounts.
N-REQ-7	Medium	Front-end and back-end code should be written with readability and maintainability in mind.
N-REQ-8	Medium	Database should support placeholder entries for testing purposes that won't appear for users with insufficient privileges.
N-REQ-9	Low	Database should support different types of media rather than only books.
N-REQ-10	Medium	Full database backups should be performed weekly.
N-REQ-11	High	Frontend updates should occur only on Saturdays at 2:00 a.m. EST.

User Interface Requirements

No.	Priority Weight	Description	Graphic
UI-REQ-1	High	The website should be neat and organized.	
UI-REQ-2	High	Once logged in, each user should see specific options available to them according to their privileges.	

UI-REQ-3	Medium	No backend or database operations should be overtly visible to the user.	
UI-REQ-4	Medium	UI changes should be implemented based on consensus from users and council of IT department.	
UI-REQ-5	High	All essential UI components should be screen-readable (including alt text for images, default text for errors, transcripts for videos).	
UI-REQ-6	Low	UI components should scale based on monitor resolution.	

Plan of Work

- Weeks 1-2
 - o Conceptualize and complete the system proposal.
- Weeks 3-4 (**We are here.**)
 - o Determine connection between frontend, backend, and database and start construction.
 - **Going well so far. My backend has switched from Java to Node.js. I've set up the MySQL schema and started on the HTML for the site. Connecting all three is proving a little confusing, but I'll figure it out.**
- Weeks 5-7

- Build database to organize books and their attributes, build frontend webpage for users to interact with, build backend to connect the two.
- Week 8
 - Test system with sample entries and actions and record demo for midterm.
- Weeks 9-11
 - Improve system with feedback from professor. Implement suggested features and bug fixes.
- Weeks 12-14
 - Write test cases and optionally add some cool last-minute features.
- Week 15
 - Test final system and record demo for final.

Functional Requirements Specification

Logan Young

Use Case Specifications

Stakeholders

1. Library staff – helpful tool to compile information on their library's stock and people who use the library
2. Library members – allows them to see what books are in stock and perform a variety of useful actions
3. Library administrators – allows them to perform administrative duties automatically and/or digitally that would otherwise be time-consuming
4. Library board of trustees – allows them to gauge interest in the library and better understand their goals
5. Local governments – a better functioning library helps to better educate the populace and the populace in turn improves their community and their government

Actors and Goals

Primary Actors:

1. Library member: This actor checks out books, marks books as returned, and places holds.
2. Library staff: This actor manages member accounts and book statuses.
3. Library administrator: This actor manages the entire system, holding the highest privileges.

Secondary Actors:

1. System: This stores information about library users, contents, etc. and performs operations.

Use Cases

Library Member (Total: 14)

1. Check out book: To check out a book from the library. (2)
2. Return book: To return a book to the library. (2)
3. Hold request: To request a hold on a book from the library. (1)
4. Procurement request: To request a book from a different library in local network. (3)
5. Delete self: To delete a user's own account from the LMS. (2)
6. Submit suggestion: To submit a suggestion for a new book to the library. (1)
7. Donate book: To donate a book to the library. (3)

Library Staff (Total: 11)

1. Hide book: To mark a book as unavailable. (1)
2. Delete account: To delete a member's account. (2)
3. Create account: To create a new account for a member. (3)
4. Adjust book: To adjust a book's attributes. (3)
5. Create book: To create a new book. (2)

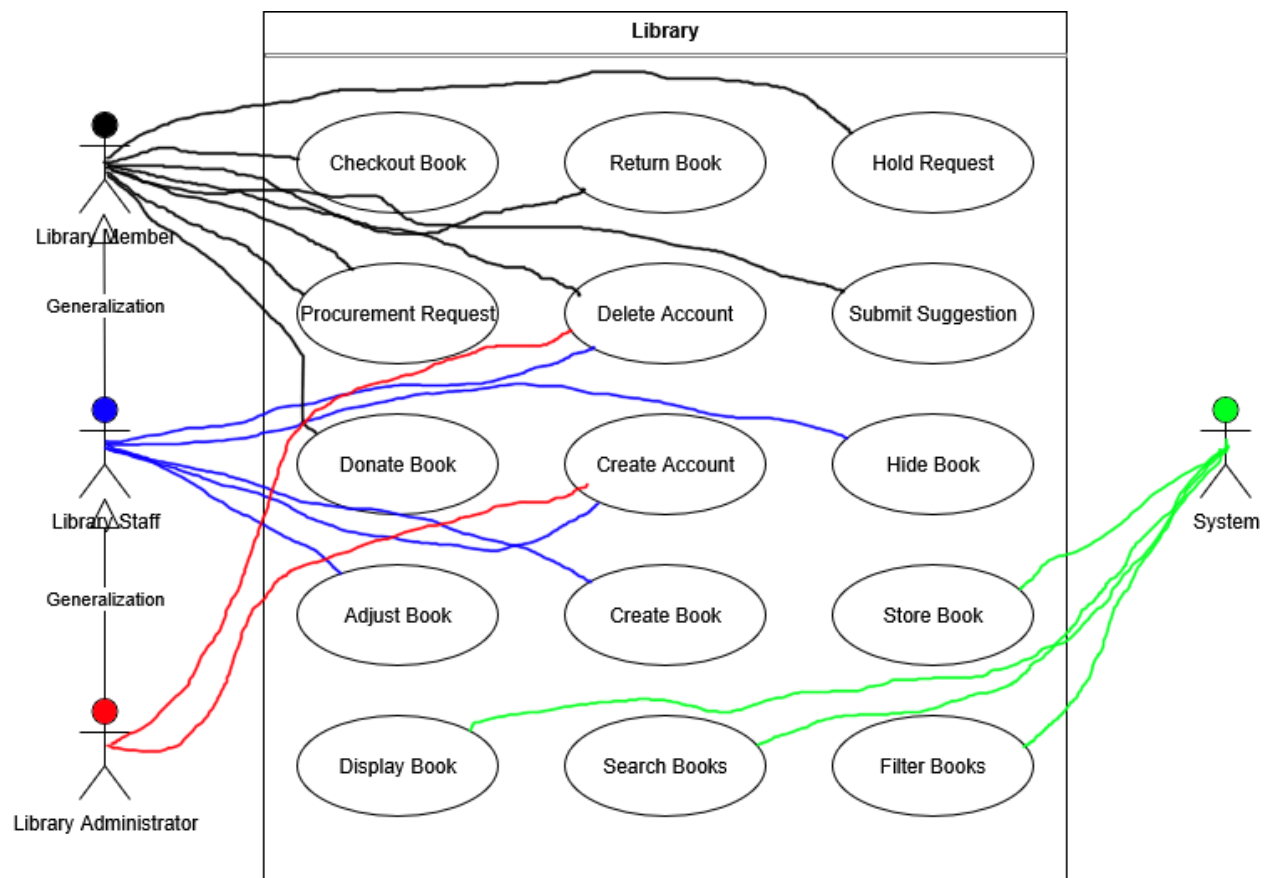
Library Administrator (Total: 4)

1. Create staff account: To convert a member account into a staff account. (2)
2. Delete account: To delete any level account (excepting administrator). (2)

System (Total: 14)

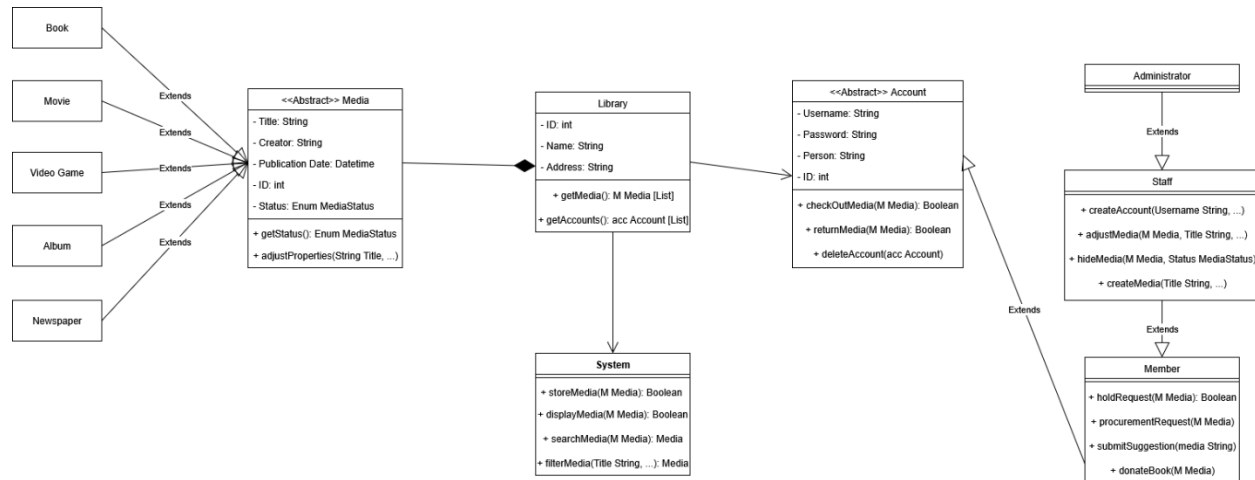
1. Store book: To store a new book in database. (3)
2. Display book: To display a new book from database. (3)
3. Search book: To search through database for a book matching criteria. (4)
4. Filter books: To filter database via matching criteria. (4)

Use Case Diagram



Class Specifications

Class Diagram



(This is a rather bare-bones class diagram. As I get further along with implementation, this is subject to change. I have a tendency to scope-creep all of my projects.)

Class Diagram Explanations

Media

Media is an abstract class since “media” isn’t itself a thing, and more of a category. It’s the template for things like books and movies. If this was a more in-depth project, each implementation of media would have its own attributes. (They still might; I haven’t made up my mind yet.) Media attributes are self-explanatory. Status refers to whether the media is “available”, “checked out”, “reserved”, or otherwise “unavailable”. It has a few basic methods right now, one to check the book’s status for the site and one to adjust properties via library staff.

Library

Library is the base for pretty much everything. It’s got a name, address, and ID since it’s planned to be able to request a piece of media from a networked library. You can get the list of Media and Accounts from the library. The accounts access the library, as well as the system.

System

The system has no attributes right now, but it does have several methods, mainly for use with the website (the user interface). The system can store media, display media, search for specific media, and filter media via provided criterion.

Account

Account is an abstract class like Media. There are a few basic attributes and methods that should be provided to all implementations. Unlike Media, there’s a hierarchy to things with Member at the bottom and Administrator at the top. Administrator can do anything that any account class below it can do.

Sequence and Activity Diagrams

Logan Young

System Sequence Diagrams

Hold Request

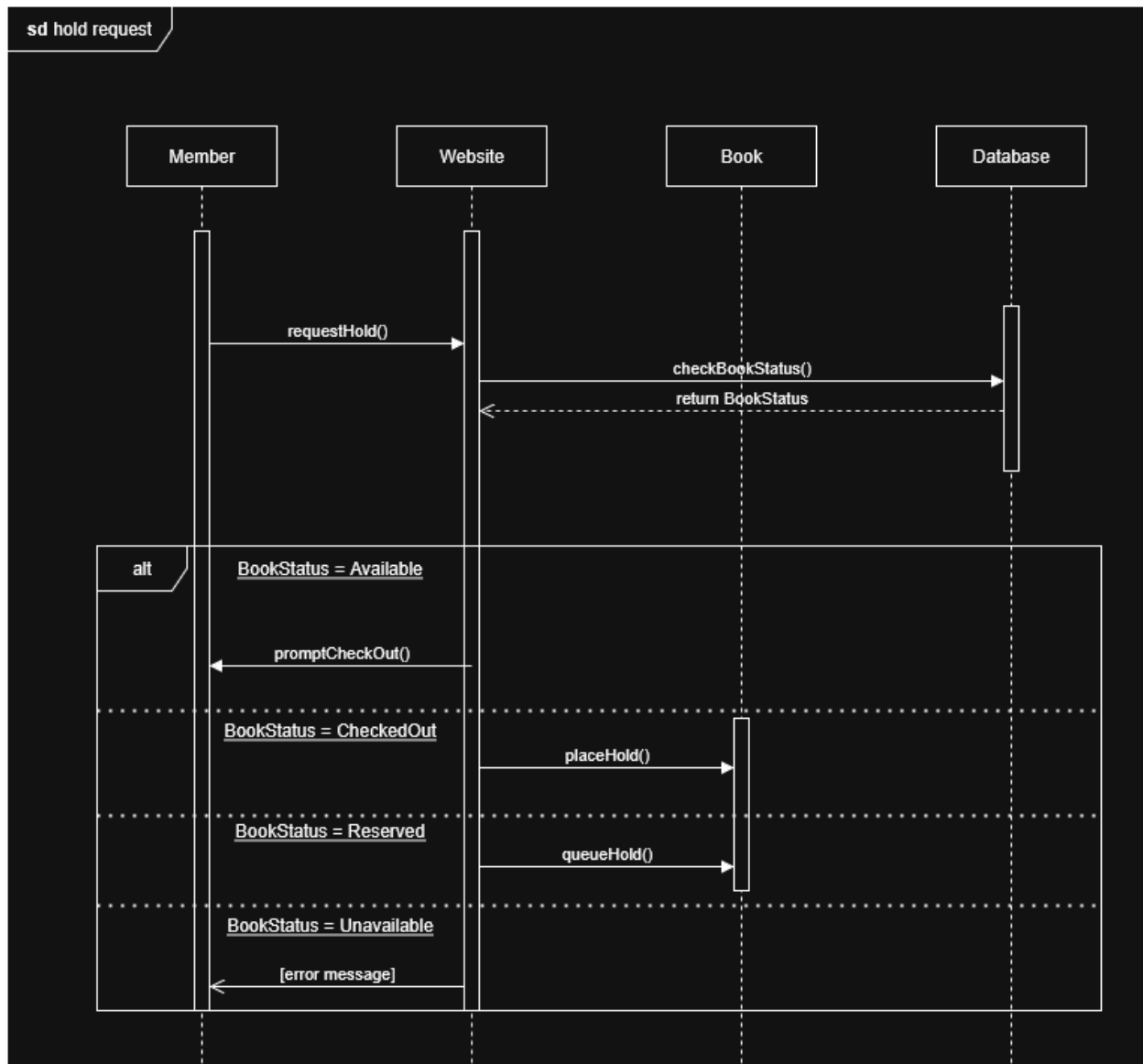
Actor and Objects

- Actor: **Member**
- Objects:
 - o Website
 - o Library Database
 - o Media

Process

1. A library member requests a hold on a book they want to borrow.
2. The system checks the book's current state: if it's available, if it's checked out, if it's reserved by another member, or if it's unavailable.
3. If the book is available: the member receives a prompt to check out the book now.
4. If the book is checked out with no other holds: the hold is placed successfully.
5. If the book is reserved by at least one other member: the member is added to the hold queue.
6. If the book is unavailable: the hold fails and an error message is sent.

Diagram



Adjust Book

Actors and Objects

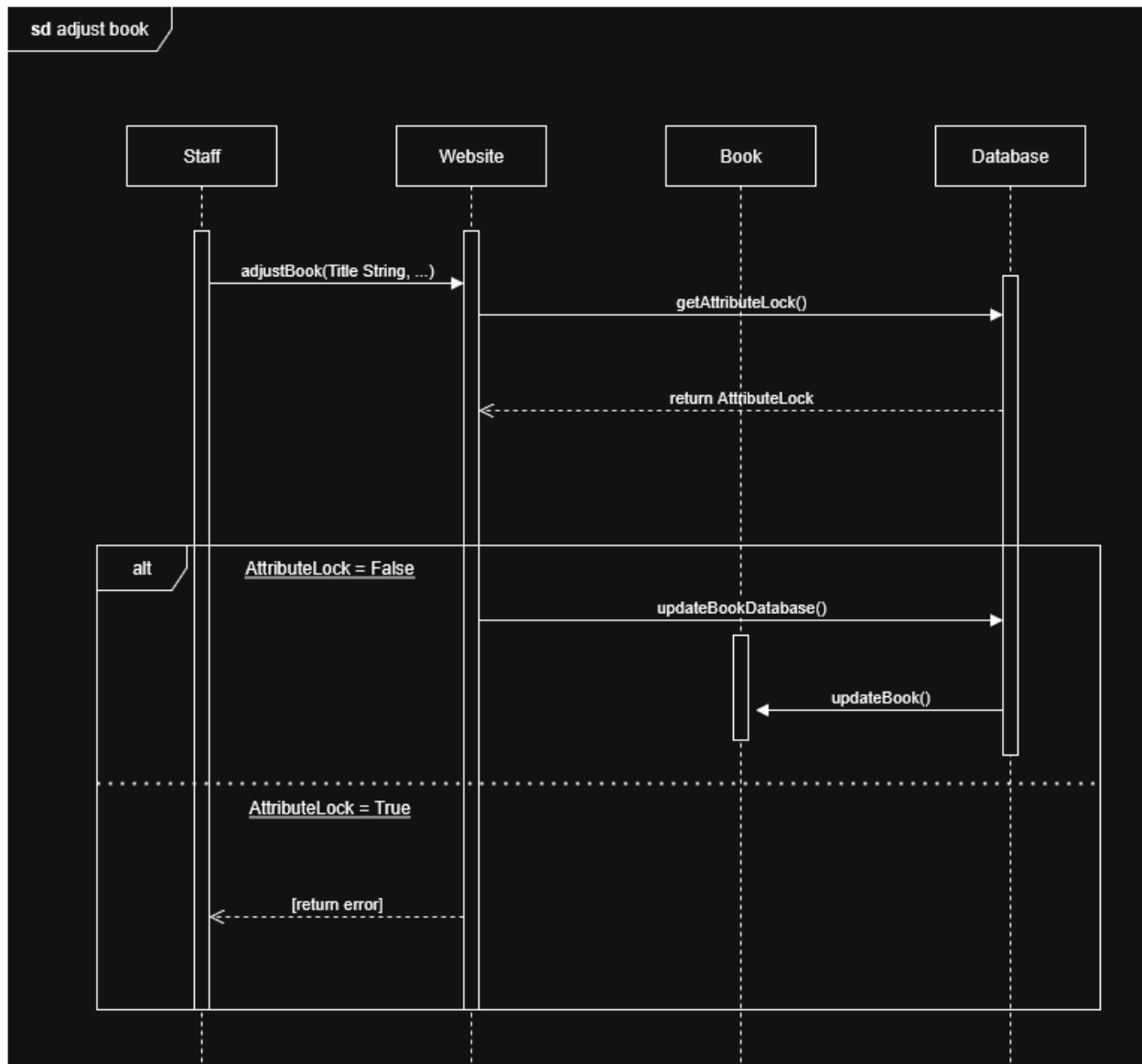
- Actor: **Staff**
- Objects:
 - o Website
 - o Library Database
 - o Media

Process

1. Staff selects a book from website to alter its attributes.

2. Staff inputs changed attributes.
3. Website checks attributeLock status.
4. If attributeLock is false: Website send changed attributes to Database, and Database stores them.
5. If attributeLock is true: Website return error and process is halted.

Diagram



Activity Diagrams

Create Staff Account

States

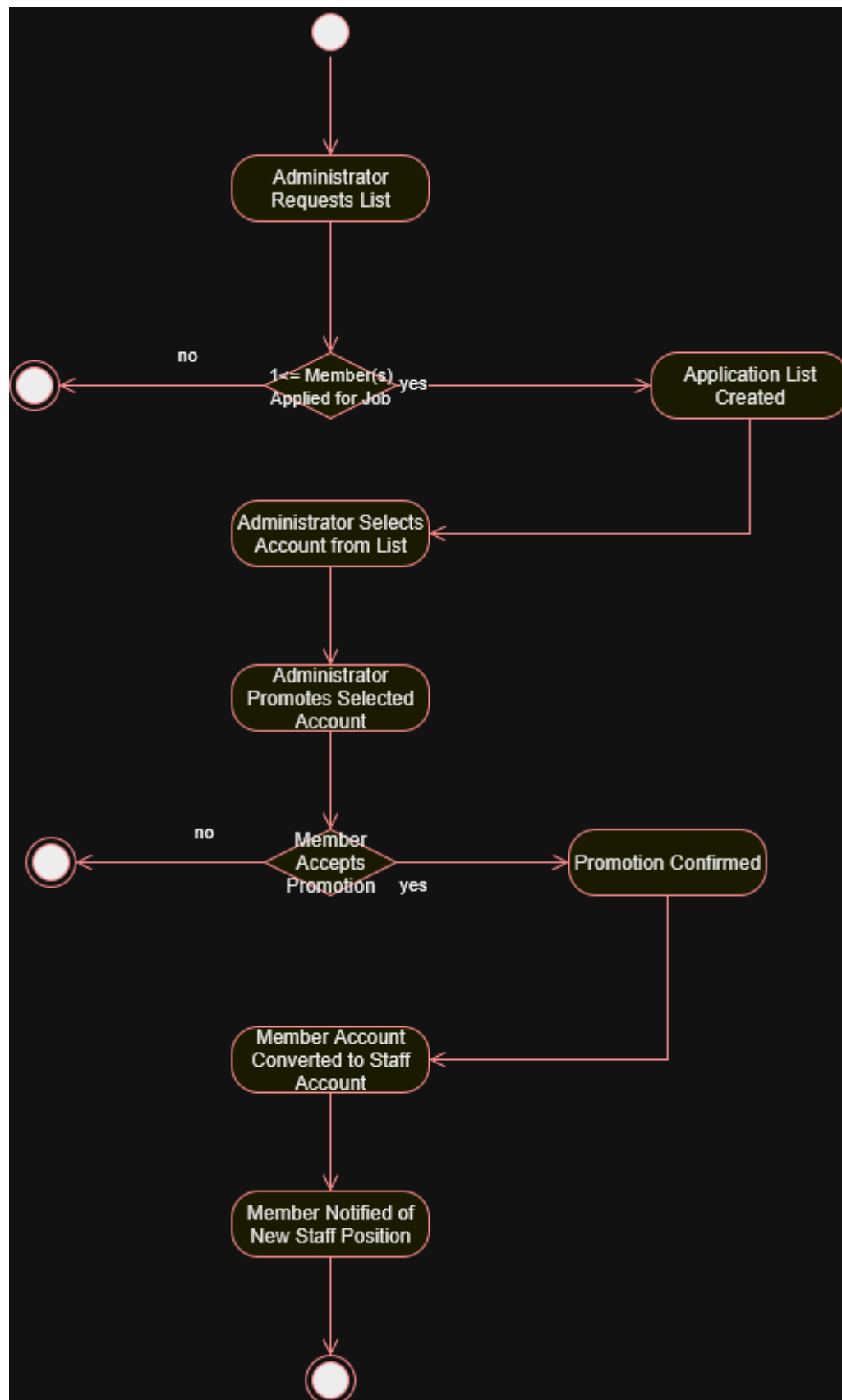
- Initial State: A member account is ready to be converted into a staff account.

- Final State:
 - No member accounts have sent in staff applications. Creation failed.
 - Member does not accept new staff position. Creation failed.
 - A member account becomes a staff account and gains access to its permissions.

Actions

1. The website retrieves a list of member accounts from the database that have applied for a staff position.
2. The Administrator selects a specific member account from the list.
3. The Administrator promotes the member account to a staff account.
4. The database converts the member account into a staff account.
5. The member is notified of their new position as a staff account.

Diagram



Filter Books

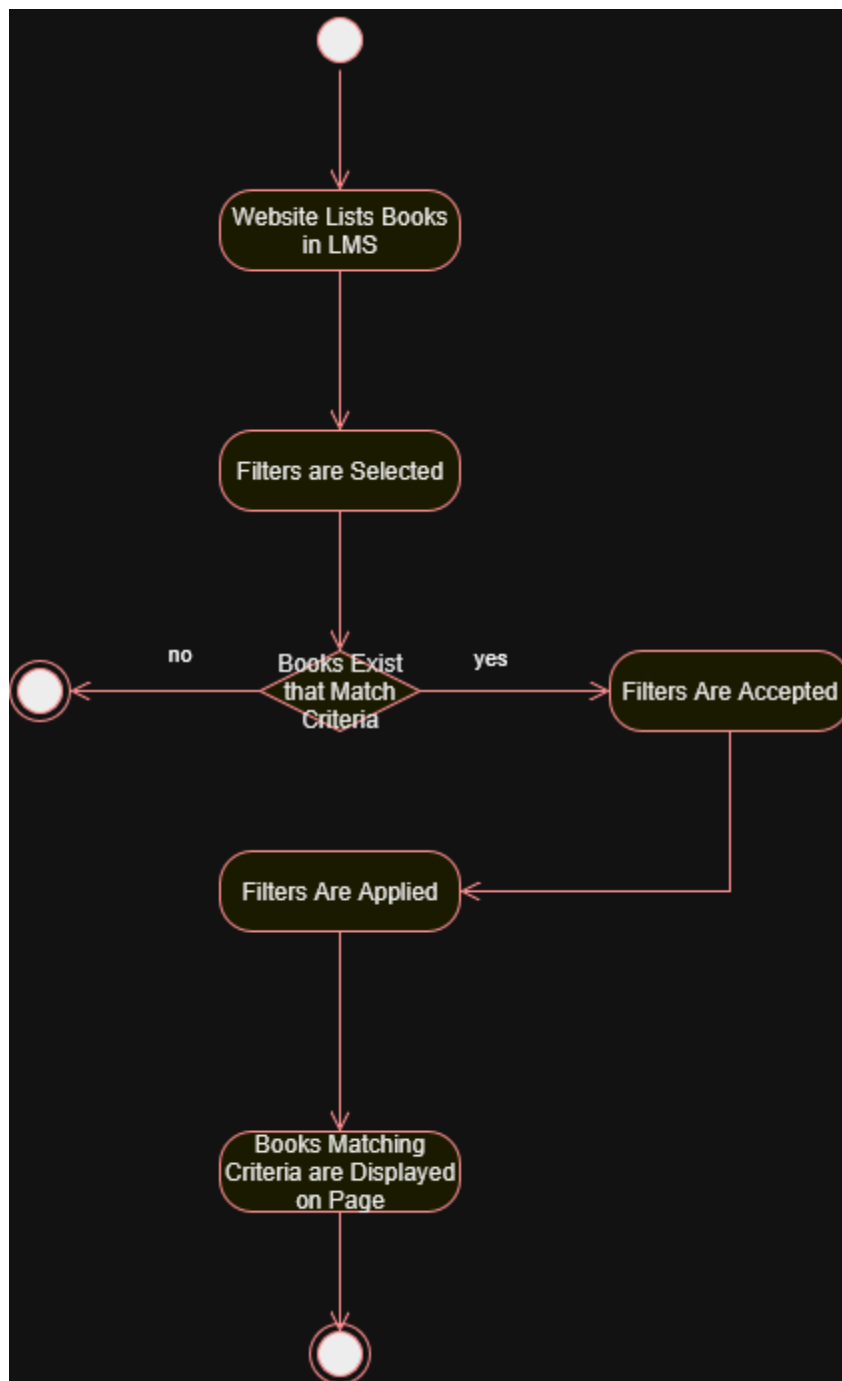
States

- Initial State: An account selects filters to apply to the book list on the website.
- Final State:
 - Books matching the filters selected are shown.
 - The page is empty to show that there are no books that match the criteria.

Actions

1. The website shows a list of books available at the library.
2. An account selects filters to apply to the list of books.
3. Filters are applied, selecting only the books that match the input criteria.
4. The books are shown on the website.

Diagram



User Interface Specification

Logan Young

Preliminary Design

Use Case 1

- A library member checks out a book.

1. Home page.

Library Management System	
Home	
User View Welcome, User! Account Level: Member Books Currently Checked Out: 4, 8, 15 Check Out Select books to check out in Book View. Check Out Return Select books to return in Book View. Return Suggest Book Author Name: Author Name Book Title: Book Title Suggest	Book View All BooksYour Books Book 1Book 2Book 3 Book 4Book 5Book 6 Book 7
Designed by Logan Young	

2. Select book.

Library Management System	
Home	
User View Welcome, User! Account Level: Member Books Currently Checked Out: 4, 8, 15 Check Out Select books to check out in Book View. Check Out Return Select books to return in Book View. Return Suggest Book Author Name: Author Name Book Title: Book Title Suggest	Book View All BooksYour Books Book 1Book 2Book 3 Book 4Book 5Book 6 Book 7
Designed by Logan Young	

3. Select check out button.

The screenshot shows the 'Library Management System' Home page. The layout is divided into two main columns: 'User View' on the left and 'Book View' on the right. The 'User View' column contains a welcome message, account level, checked-out books, and buttons for 'Check Out', 'Return', and 'Suggest Book'. The 'Check Out' button is highlighted with a red circle. The 'Book View' column shows a grid of books, with 'Book 5' circled in red. The footer indicates the design is by Logan Young.

Library Management System
Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Check out selected book?

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books Your Books

Book 1 Book 2 Book 3

Book 4 Book 5 Book 6

Book 7

Designed by Logan Young

4. Confirmation.

The screenshot shows the 'Library Management System' Home page with a confirmation dialog box. The dialog box is white with a black border and contains the text 'Checkout Successful!' and a 'Confirm' button. The background is slightly dimmed. The 'User View' and 'Book View' columns are visible behind the dialog box. The footer indicates the design is by Logan Young.

Library Management System
Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

Book 2 Book 3

Book 5 Book 6

Checkout Successful!

Confirm

Designed by Logan Young

Use Case 2

- A library member returns a book and then views their current checked-out books.

1. Home page.

Library Management System	
Home	
User View Welcome, User! Account Level: Member Books Currently Checked Out: 4, 8, 15 Check Out Select books to check out in Book View. Check Out Return Select books to return in Book View. Return Suggest Book Author Name: Author Name Book Title: Book Title Suggest	Book View All Books Your Books Book 1Book 2Book 3 Book 4Book 5Book 6 Book 7
Designed by Logan Young	

2. Select Your Books section.

Library Management System	
Home	
User View Welcome, User! Account Level: Member Books Currently Checked Out: 4, 8, 15 Check Out Select books to check out in Book View. Check Out Return Select books to return in Book View. Return Suggest Book Author Name: Author Name Book Title: Book Title Suggest	Book View All Books Your Books Book 1Book 2Book 3 Book 4Book 5Book 6 Book 7
Designed by Logan Young	

3. Select a book to return.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books Your Books

Book 1 Book 2 Book 3

Designed by Logan Young

4. Click return button.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Return book?

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books Your Books

Book 1 Book 2 Book 3

Designed by Logan Young

5. Confirmation.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

Book 2

Book 3

Return Successful!

Confirm

Designed by Logan Young

6. Return to book view.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books Your Books

Book 1

Book 2

Designed by Logan Young

Use Case 3

- A user filters books by selected criteria.

1. Home page.

Library Management System	
Home	
User View Welcome, User! Account Level: Member Books Currently Checked Out: 4, 8, 15 Check Out Select books to check out in Book View. Check Out Return Select books to return in Book View. Return Suggest Book Author Name: Author Name Book Title: Book Title Suggest	Book View All Books Your Books Book 1Book 2Book 3 Book 4Book 5Book 6 Book 7
Designed by Logan Young	

2. Select filter menu (will appear as expandable menu below Books sections).

Library Management System	
Home	
User View Welcome, User! Account Level: Member Books Currently Checked Out: 4, 8, 15 Check Out Select books to check out in Book View. Check Out Return Select books to return in Book View. Return Suggest Book Author Name: Author Name Book Title: Book Title Suggest	Book View All Books Your Books Book 1Book 2Book 3 Book 4Book 5Book 6 Book 7 Filter List Title contains: Author contains: Publication date: XX-XX-XXXX ☐ Has digital copy☐ Has audiobook ☐ Has physical copy☐ Is hardcover
Designed by Logan Young	

3. Choose filters.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books

Your Books

Book 1

Book 2

Book 3

Book 4

Book 5

Book 6

Book 7

Filter List

Title contains:

Author contains:

Publication date:

☐ Has digital copy

☐ Has audiobook

☒ Has physical copy

☐ Is hardcover

Apply

Designed by Logan Young

4. Click apply.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books

Your Books

Book 1

Book 2

Book 3

Book 4

Book 5

Book 6

Book 7

Filter List

Title contains:

Author contains:

Publication date:

☐ Has digital copy

☐ Has audiobook

☒ Has physical copy

☐ Is hardcover

Apply

Designed by Logan Young

5. Filtered books.

Library Management System

Home

User View

Welcome, User!

Account Level: Member

Books Currently Checked Out: 4, 8, 15

Check Out

Select books to check out in Book View.

Check Out

Return

Select books to return in Book View.

Return

Suggest Book

Author Name:

Book Title:

Suggest

Book View

All Books Your Books

Book 2

Book 5

Filter List

Title contains:

Author contains:

Publication date: XX-XX-XXXX

☐ Has digital copy

☒ Has physical copy

☐ Has audiobook

☐ Is hardcover

Apply

Designed by Logan Young

Use Case 4

- A staff member adds a book.

1. Home page.

Library Management System

Home

User View

Welcome, User!

Account Level: Staff

Books Currently Checked Out: 4, 8, 15

Member Staff

Create Book

Open Form

...

...

Book View

All Books Your Books

Book 1

Book 2

Book 3

Book 4

Book 5

Book 6

Book 7

Designed by Logan Young

2. Select Open Form.

Library Management System

Home

User View

Welcome, User!

Account Level: Staff

Books Currently Checked Out: 4, 8, 15

Member

Staff

Create Book

Open Form

...

...

Book View

All Books

Your Books

Book 1

Book 2

Book 3

Book 4

Book 5

Book 6

Book 7

Designed by Logan Young

3. Fill out form for book creation.

Library Management System

Home

User View

Welcome, User!

Account Level: Staff

Books Currently Checked Out: 4, 8, 15

Member

Staff

Create Book

Open Form

...

...

Book Creation Form

Title: Stars and Space 3: More Space

Author: Jessica Spacey

Publication Date: 01-23-1997

Format: Physical, Softcover

Create

Book View

Book 2

Book 3

Book 5

Book 6

Designed by Logan Young

4. Select Create.

The screenshot displays the 'Library Management System' home page. The layout is divided into three main sections: 'User View' on the left, a central content area, and 'Book View' on the right. The 'User View' section includes a welcome message, account level ('Staff'), checked-out books, and a 'Create Book' button labeled 'Open Form'. The central content area features a 'Book Creation Form' with fields for Title, Author, Publication Date, and Format, each with a red oval highlighting the input area. Below the form is a 'Create' button, also highlighted with a red oval. The 'Book View' section shows a list of books. The footer contains the text 'Designed by Logan Young'.

Library Management System
Home

User View

Welcome, User!

Account Level: Staff

Books Currently Checked Out: 4, 8, 15

Member **Staff**

Create Book

Open Form

...

...

Book Creation Form

Title: Stars and Space 3: More Space

Author: Jessica Spacey

Publication Date: 01-23-1997

Format: Physical, Softcover

Create

Book View

Book 2

Book 3

Book 5

Book 6

Designed by Logan Young

5. Confirmation.

The screenshot displays the 'Library Management System' home page after a successful book creation. The layout is identical to the previous screenshot, but the central content area now shows a 'Book Creation Successfull' message with a 'Confirm' button at the bottom. The 'User View' and 'Book View' sections remain unchanged. The footer contains the text 'Designed by Logan Young'.

Library Management System
Home

User View

Welcome, User!

Account Level: Staff

Books Currently Checked Out: 4, 8, 15

Member **Staff**

Create Book

Open Form

...

...

Book Creation Successfull

Confirm

Book View

Book 2

Book 3

Book 5

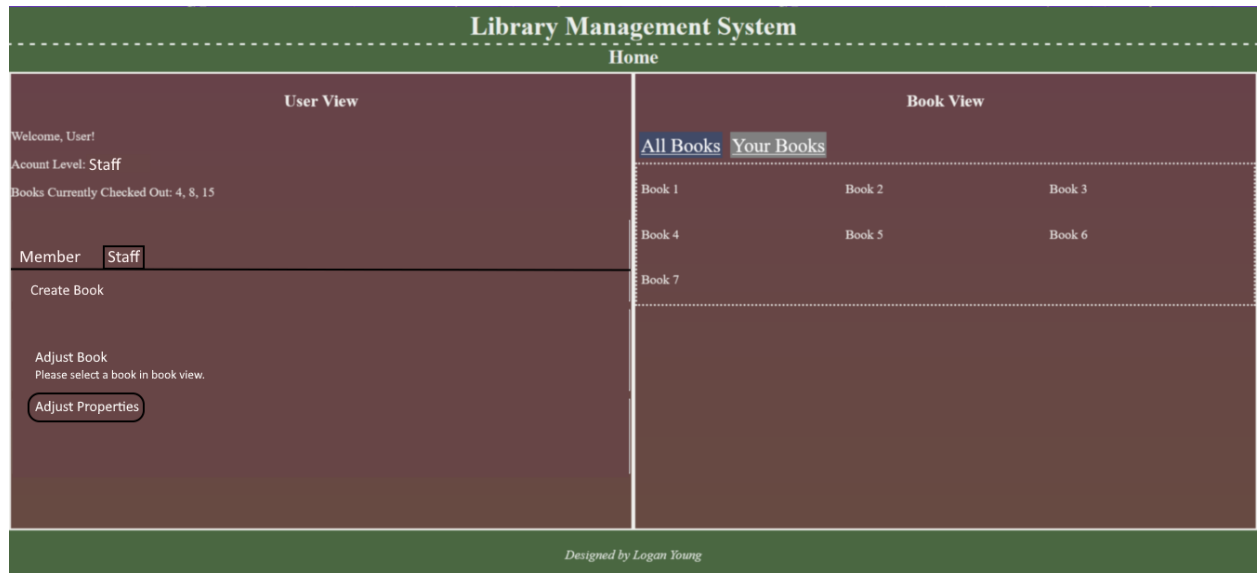
Book 6

Designed by Logan Young

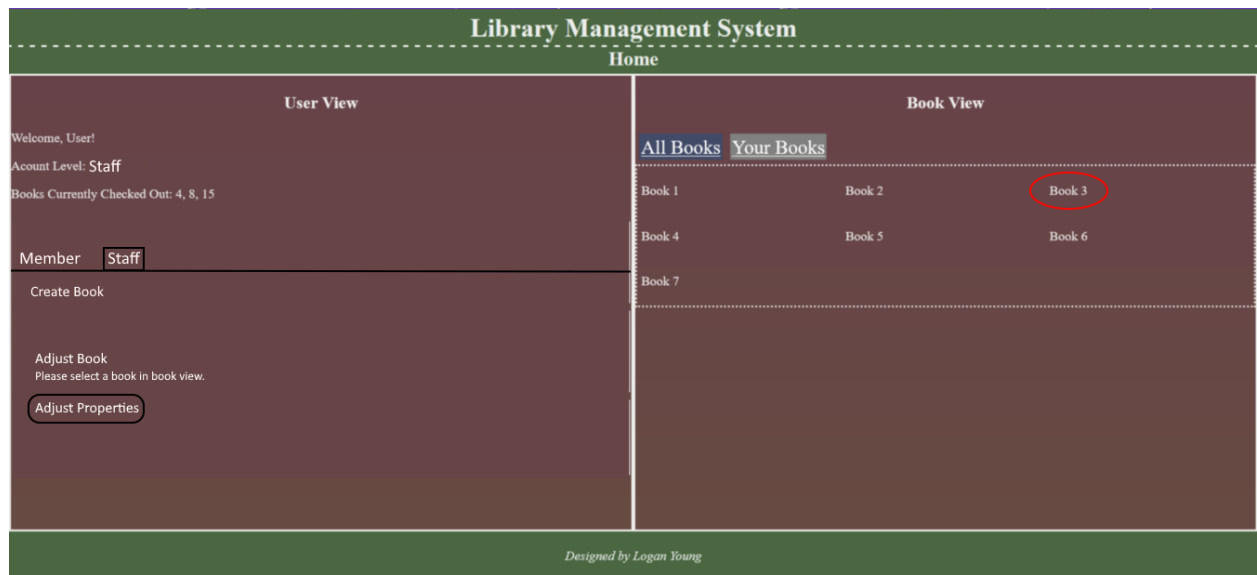
Use Case 5

- A staff member adjusts a book's properties.

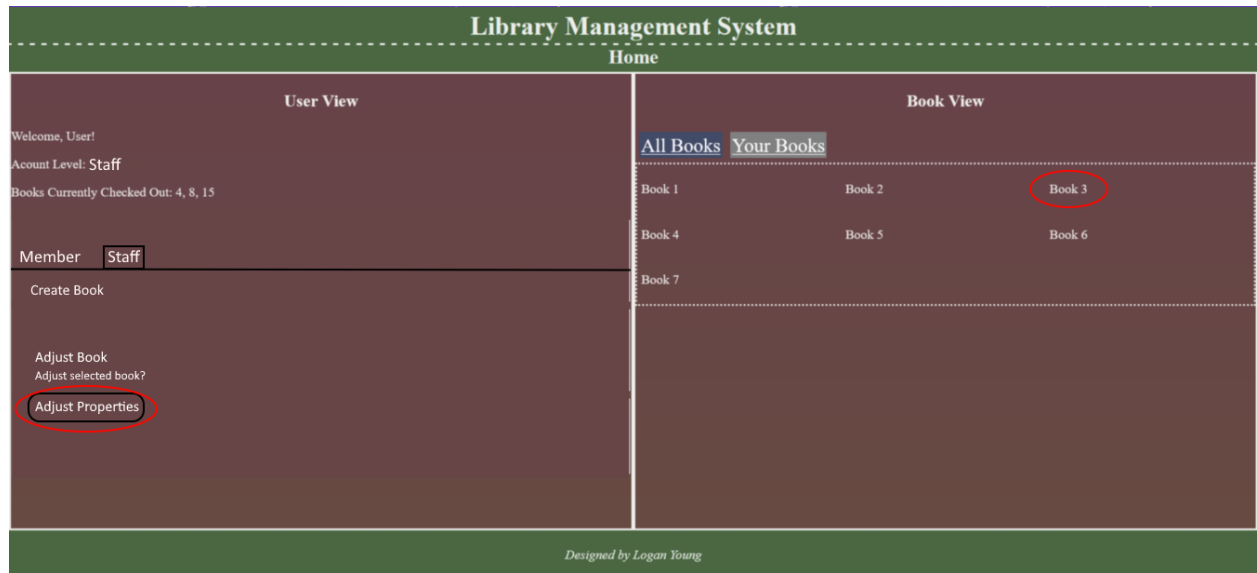
1. Home page.



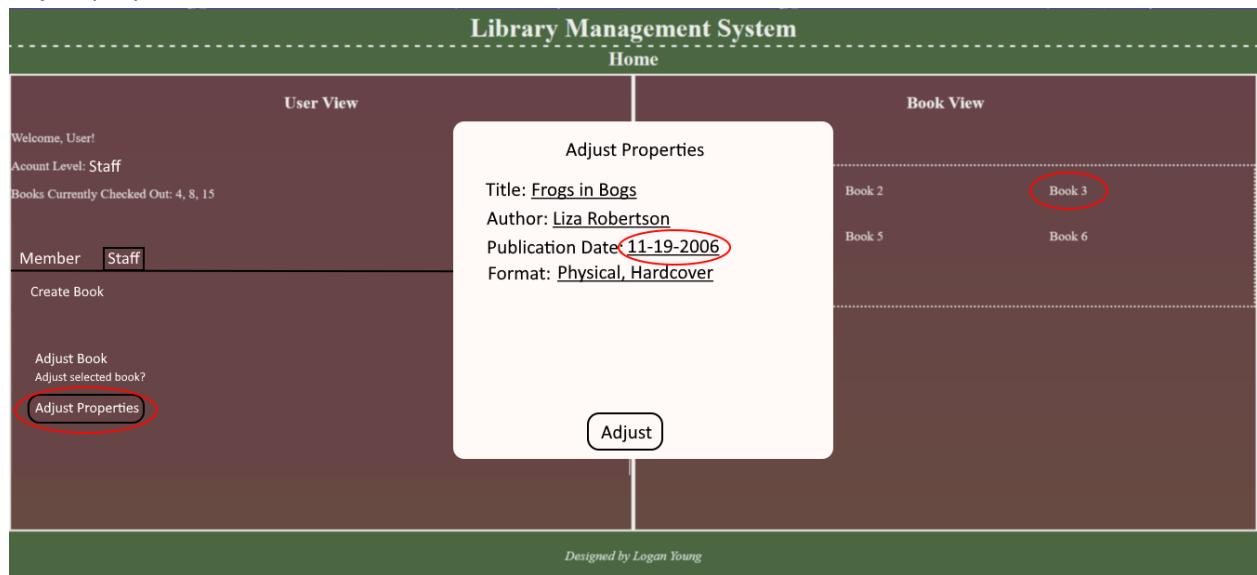
2. Select book.



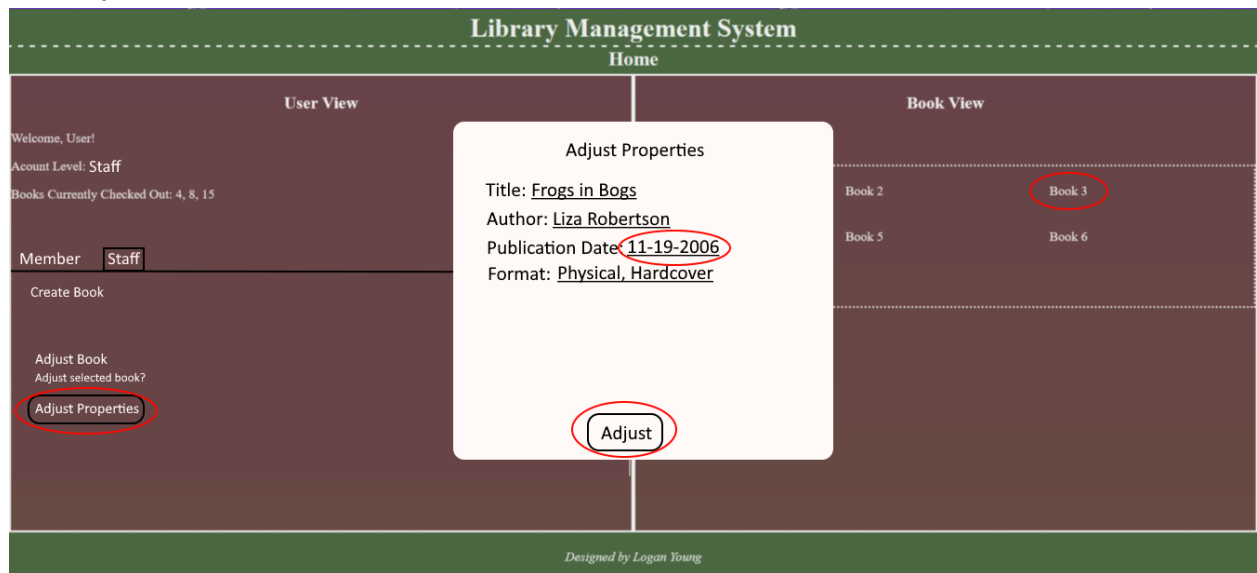
3. Click Adjust Properties.



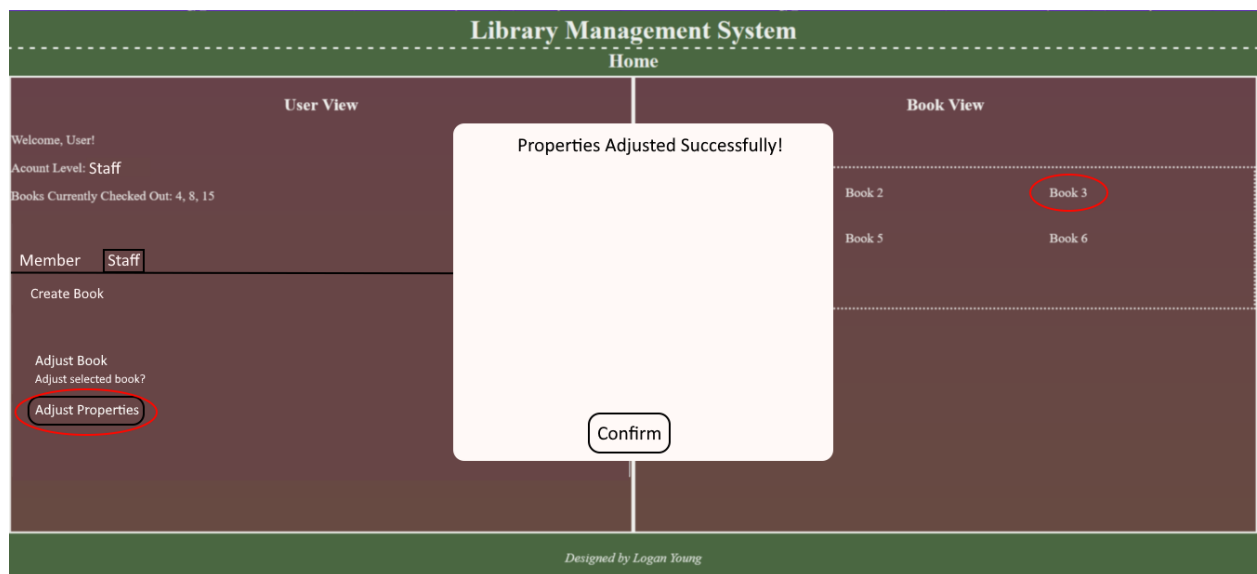
4. Adjust properties in form.



- Click Adjust.



- Confirmation.



User Effort Estimation

Usage Scenario	Navigation	Clicks	Keystrokes
Member checks out a book	Home -> Book View -> User View -> Confirmation	3	0
Member returns a book and views current checked out books	Home -> Book View-> Confirmation -> Home -> Book View	5	0
Staff adjusts a book's properties	Home -> Book View -> User View -> Popup Form -> Confirmation	>=5	100>=x>=1

Staff adds a book	Home -> User View -> Popup Form -> Confirmation	>=4	100>=x>=10
User filters books	Home -> Book View	>=3	>=0

Additional Notes

- Please excuse the poor photoshopping done on top of my website screenshots.
- Like the sample example, each use case assumes that the user has already signed in. Below is an early work-in-progress example of the sign in form.

Library Management System

Sign In

Sign In Form

Account Type: Member ▼

Email:

Password:

Designed by Logan Young

- The base home page and sign in page are currently being worked on, but they are fully coded in HTML and CSS. JavaScript and node.JS functionality are coming soon. Might support JavaScript frontend with jQuery as I progress.
- The appearance of the site and UI placement are subject to change, but the general functionality will remain the same.

Traceability Matrix and Updates

Logan Young

Updated Functional Requirements

No.	Priority Weight (1-5; 1: lowest)	Description
REQ-1	5	The LMS should be able to store and display books on the website.
REQ-2	5	Library members should be able to use the LMS to check out and return books.
REQ-3	3	Library members should be able to use the LMS to submit a hold request for a maximum of three books.
REQ-4	4	Library staff should be able to use the LMS to mark a book as unavailable.
REQ-5	3	Library members should be able to delete their account from the LMS.
REQ-6	4	Library staff should be able to delete a member's account using the LMS.
REQ-7	5	Library staff should be able to create a new member account for a user.
REQ-8	5	Library staff should be able to adjust a book's attributes if incorrect.
REQ-9	5	Library staff should be able to create a new entry in the LMS for a newly received book.
REQ-10	2	Library members should be able to request a book from a different library in the same network.
REQ-11	4	Library administrator should be able to convert a new user into staff.
REQ-12	4	Library administrator should be able to convert staff into members or delete accounts entirely.
REQ-13	2	Library members should be able to submit a suggestion for new books for the library.
REQ-14	1	Library members should be able to create a new donated book entry to be approved by staff.
REQ-15	5	LMS users should be able to easily search for a specific book using various filters and sorting techniques.
REQ-16	5	LMS users should be able to log in to personal accounts using email and passwords from the website and log out.
REQ-17	2	LMS users should be able to activate 2-factor authentication for their accounts from user settings.
REQ-18	3	Library administrators should be able to completely delete a book from the database.

Updated Use Cases

No.	Name	Description
UC-1	Log In	To let a user log in to their account.

RE Q3	3	X					X												
RE Q4	4	X														X			
RE Q5	3	X							X										
RE Q6	4	X							X										
RE Q7	5	X								X									
RE Q8	5	X													X				
RE Q9	5	X												X					
RE Q1 0	2	X						X											
RE Q1 1	4	X									X								
RE Q1 2	4	X							X	X	X								
RE Q1 3	2	X										X							
RE Q1 4	1	X											X						
RE Q1 5	5	X																X	X
RE Q1 6	5	X	X																
RE Q1 7	2	X		X															
RE Q 18	3																X		
Ma x PW		5	5	2	5	5	3	2	4	5	4	2	1	5	5	4	3	5	5
Tot al PW		5 9	5	2	5	5	3	2	1 1	9	8	2	1	5	5	4	3	5	5

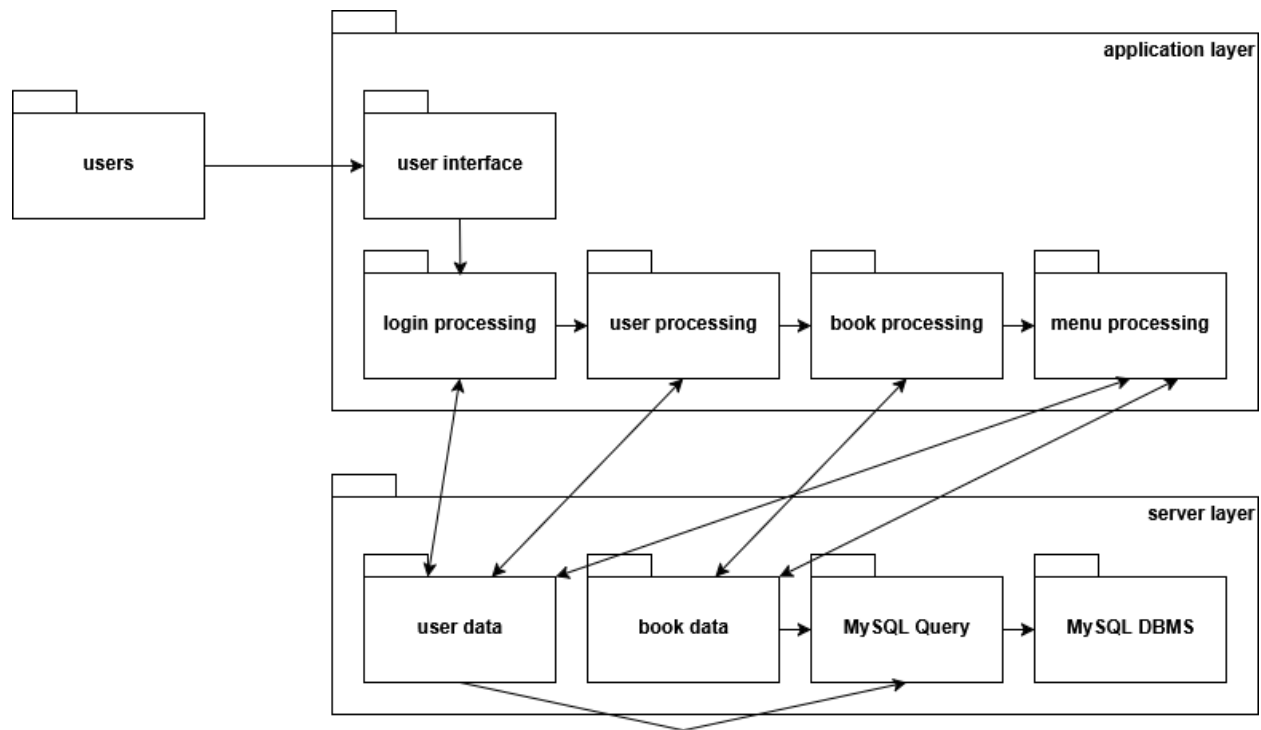
System Architecture and System Design

Architectural Styles

The Library Management System uses a client-server architectural style. The client is a computer operated by an end user, and the server is a computer solely used for running the website. The server would be located in the real library. I'm using a combination of HTML, CSS, and JS for the frontend; NodeJS and Express for the backend; and MySQL for the database. (I'm currently deciding on another language to send information from the client to the server.) The frontend displays the website and allows the user to perform some actions to solicit a response. The backend communicates between the frontend and the database. The database (MySQL) stores all information used for the application: user data, book data, etc. As of now, both the client and the server run on the same computer when you use the application, but in production, the client would be on a separate device from the server (which would be running from the library).

Identifying Subsystems

Shown below is a UML package diagram of the Library Management System. The application layer represents the frontend, while the server layer represents the backend and database. The process begins with the user accessing the user interface package. From there, the user interface package contains four processes (what could also be defined as subsystems): login processing, user processing, book processing, and menu processing. Login processing accesses user data in the server layer and controls how the user logs in and out. User processing accesses user data and controls the various aspects of user account details. Book processing accesses book data and controls all book operations. Menu processing accesses both user data and book data and controls UI displays and changes. The data in the server layer are modified after querying the MySQL database.



Mapping Subsystems to Hardware

All subsystems are run on one server computer and one client computer.

Persistent Data Storage

There are two main objects that are stored globally: users and books. Both of these are stored as relations in a MySQL database, saved to the server. Other secondary objects are also stored as relations, such as “suggested books”.

Network Protocol

There will only be one server computer.

Global Control Flow

Execution Orders

My system is event-driven, that is, users can perform different actions in different orders from other users. The LMS does this through forms and buttons. For example, one user could check out a book and then suggest a book, while another user could return a book with no further actions. In standard operation, a user would click the icon of a book in the “Book View” and then use the “User View” to check out that selected book.

Time Dependency

The LMS is event-driven, and my implementation does not depend on real time or any sort of timer. However, one potential feature is a “Book of the Day”, which would advertise a random book from the database each day. This would change in real time, most likely at midnight in the server’s time zone. Other than that, the only waiting is done in periods before the user inputs some action.

Concurrency

The LMS is single threaded.

Hardware Requirements

- Most components of the user interface are scrollable, so only a standard resolution of 640x480 pixels is required.
- The user needs a device with a user interface capable of connecting to the internet.
- 1 GB of memory should be sufficient for processing operations.
- 10 Mbps upload/download network speed should be sufficient to send and receive data from the LMS.

User Interface Design and Implementation

The final user interface was designed with ease of use in mind, not aesthetics. A dual-pane window is used for the main page so that user and book tasks are separated. For some tasks, the user can select a book in the book view and then perform some operation in the user view.

Library Management System

Home

Log Out

User View

Book View

Welcome, test!

Account Level: MEMBER

Books Currently Checked Out:

Hitchhiker's Guide to the Galaxy by Douglas Adams

Rendezvous with Rama by Arthur C. Clarke

Check-Out

Return

Suggest

Check Out

Select a book to check out in the Book View.

Check Out

All Books

Your Books

Hitchhiker's Guide to the Galaxy

Invisible Cities

The City and the City

Collected Fictions

Piranesi

The Invisible Man

The Time Machine

Rendezvous with Rama

Title: Hitchhiker's Guide to the Galaxy Author: Douglas Adams Genre: Science Fiction Status: CHECKED_OUT Publication Date: 1979-10-12T04:00:00.000Z ISBN: 9780330258647 Local ID: 1

Filters

Title Contains:

Author Contains:

Genre Contains:

Availability Is:

[Any]

Sort

Sort by Title in Ascending order.

Designed by Logan Young

Library Management System

Sign In

Sign In

Account Type: Member

Email: test@test.com

Password:

Submit

Create Account

First Name: Your First Name

Last Name: Your Last Name

Email: Your Email

Password: Your Password

Submit

Designed by Logan Young

Home

Log Out

User View

Book View

Welcome, bob!

Account Level: STAFF

Books Currently Checked Out:

Collected Fictions by Jorge Luis Borges

Check-Out

Return

Suggest

Edit Book

Manage Users

Edit Book

Select a book to edit or create a new book.

Edit Book

Create Book

Edit Title:

New Title

Edit Author:

New Author

Edit Genre:

New Genre

Edit Publication Date:

1970-01-01

Edit ISBN:

0172456789012

All Books

Your Books

The Invisible Man

The Time Machine

The Masque of the Red Death

Hitchhiker's Guide to the Galaxy

The City and the City

Filters

Title Contains:

the

Author Contains:

Genre Contains:

Availability Is:

[Any]

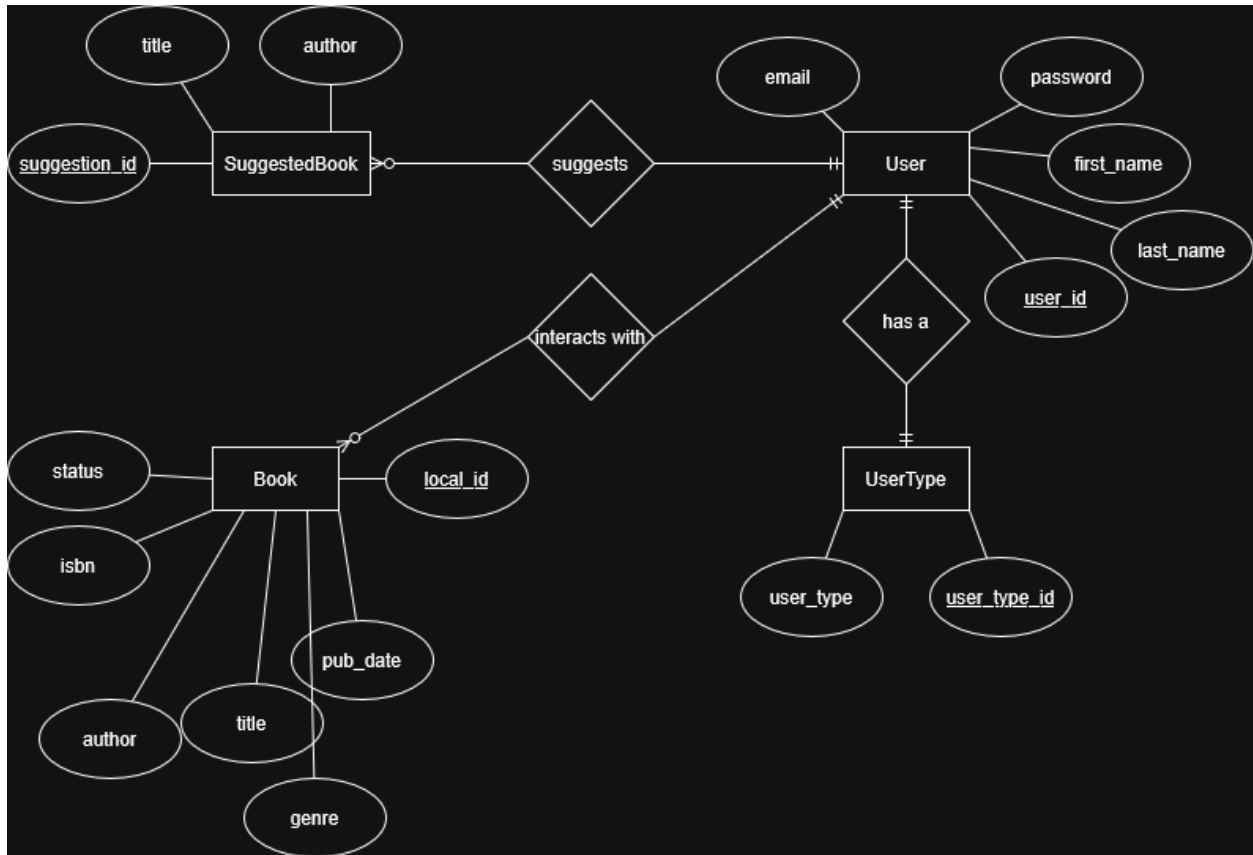
Sort

Sort by Author in Descending order.

Project Plan

- Weeks 1-2
 - Conceptualize and complete the system proposal.
- Weeks 3-4
 - Determine connection between frontend, backend, and database and start construction.
- Weeks 5-7
 - Build database to organize books and their attributes, build frontend webpage for users to interact with, build backend to connect the two.
- Week 8
 - Test system with sample entries and actions and record demo for midterm.
- Weeks 9-11
 - Improve system with feedback from professor. Implement suggested features and bug fixes. Implement rest of features left out from demo.
- Weeks 12-14
 - Write test cases and optionally add some cool last-minute features.
- Week 15 (**We are here.**)
 - Test final system and record demo for final.

Data Model



Normalization ensures cleanness and efficiency in regard to data record storage. Primarily, normalization prevents the existence of repeating groups which can have a negative effect on systems. However, normalization also helps database developers write simple, modular data relationships that can be easily updated and extended over time. Indexing, in turn, will allow data to be quickly found and retrieved once stored.

Coding Standards and UI Design

Coding standards used with the LMS will be relatively simple. Systems coding will use camel case, while database naming will use snake case. Commenting will be used throughout the system to document all functions and important variables. Each file will begin with a header comment detailing author, creation date, last update, and purpose. As far as possible with web development, each page will be modular. The backend will adhere to Object-Oriented Programming principles since it will be coded in Java.

Usability principles will be implemented where possible. Site functions should take no more than a certain amount of time and a certain number of clicks to complete. The design should not hide any important functions behind menus. Additionally, the design should be easy to read and view for people

with visual impairments. The site should be compatible with screen readers. Finally, the LMS should be tested and updated for use with multiple browser and device types.

The screenshot displays a web application for a library management system. The top navigation bar is green with the text 'Home' and a 'Log Out' link. The main content area is divided into two panels: 'User View' on the left and 'Book View' on the right.

User View:

- Greeting: 'Welcome, bob!'
- Account Level: 'STAFF'
- Books Currently Checked Out:
 - Collected Fictions by Jorge Luis Borges
- Buttons: 'Check-Out', 'Return', 'Suggest', 'Edit Book', 'Manage Users'.
- Section: 'Edit Book'
 - Instruction: 'Select a book to edit or create a new book.'
 - Form fields:
 - Edit Book: Radio buttons for 'Create Book' and 'Edit Book'.
 - Edit Title: Text input with 'New Title' placeholder.
 - Edit Author: Text input with 'New Author' placeholder.
 - Edit Genre: Text input with 'New Genre' placeholder.
 - Edit Publication Date: Text input with '1970-01-01' placeholder.
 - Edit ISBN: Text input with '0112754567890112' placeholder.

Book View:

- Tabs: 'All Books', 'Your Books'.
- Book List:
 - The Invisible Man
 - The Time Machine
 - The Masque of the Red Death
 - Hitchhiker's Guide to the Galaxy
 - The City and the City
- Filters:
 - Title Contains: Text input with 'the' placeholder.
 - Author Contains: Text input.
 - Genre Contains: Text input.
 - Availability Is: Dropdown menu with 'Any' selected.
- Sort:
 - Sort by: Dropdown menu with 'Author' selected.
 - in: Dropdown menu with 'Descending' selected.
 - order: Text input.

Testing Strategy

Testing for the LMS will cover the usual four areas: unit, integration, system, and acceptance. Unit testing will be comprised of testing each individual function for the LMS to ensure all run without error given some test data (e.g., a login script, an "add book" function). Then, all connected functions and scripts will need to be tested together, likely by inserting test data into one function and then handing the results of that function to the next (like checking out a book for the logged in user and then changing that book to unavailable in the DB). System testing will occur when the previous two tests have finished: testers will run through a complete version of the site to ensure that everything works as specified. Finally, acceptance testing will occur using the userbase as testers: library staff and members. If feedback is given on changes, these will be analyzed and possibly incorporated into the next iteration of the LMS. Most tools used will be IDEs or virtual environments; if needed, automatic tools like postman will be used to check data flows and ensure server connections are stable and working.

Test Case ID	Description	Precondition	Input	Expected Output
TC-F-011	Member attempts to check out a book that is not available.	User account is member level or higher and chosen book is not available.	- Book: Animal Farm [Checked Out]	Book checkout fails, "This book is [Checked Out], please submit a hold request or choose another book!"
TC-V-004	Email does not comply with regex email check.	User has entered an incorrect email	- Email: lmsuser1.com	Account creation disabled, email field warning, "Email is invalid! Please check to

		address into the email field.		make sure email is formatted correctly."
--	--	-------------------------------	--	--

Error Handling and Debugging

Debugging must be done before the new system is deployed, but there is also room for user testing afterward. In other words, the system must be free of large, functional bugs before deployment. Different types of errors will need different strategies: server connection errors will require a review of the server architecture and functions that connect the client to the server to the database, functional errors will need a review of the application scripts, syntax errors typically point to themselves, etc. Using an IDE rather than a word processor makes error discovery and fixing much simpler.

The following is an example of basic error handling related to the “check out book” function for the LMS:

```
try {
  // if user already has three books checked out
  if(document.getElementsByClassName("yourBook").length > 2) {
    alert("You already have 3 books checked out!");
  }
  // if user has not selected a book yet
  else if(chosen.length == 0) {
    alert("You have not selected a book to check out yet!");
  }
  else {
    book_id = chosen[0].firstElementChild.getAttribute("data-id");
    for(let book of books) {
      if(book.local_id == book_id) {
        if(!(book.status == "AVAILABLE")) {
          alert("This book is not currently available! Its status
is: " + book.status);
        }
        else if(book.status == "AVAILABLE") {
          // post request to update db
          await fetch('/checkout', {
            method: 'POST',
            body: JSON.stringify({
              "local_id": book.local_id,
              "status": "CHECKED_OUT"
            }),
            headers: {"Content-Type": "application/json"}
          });
          // additionally, update local book to pass to display

```

```

        book.status = "CHECKED_OUT"
        // set checked out flag to true
        bookCheckedOut = true;
        // break out of loop
        break;
    }
}
}
// if book check out was successful
if(bookCheckedOut == true) {
    console.log("book checked out client side!");
    //displayBooks(books);
}
}
}
}
catch (error) {
    console.error(error);
}

```

Collaboration and Code Integration

As aforementioned, version control will be implemented through GitHub. Each team member will contribute to development, but some members will be given specific assignments to help development flow better. Team members will have a weekly meeting that also serves as a code review so they can give each other feedback and work through difficult issues together.

An example of the workflow might be something like the following:

- Two of five developers are working on the main functions. They regularly work together so as to ensure they do not hamper each other's work.
- The next two developers work on the backend and connection to the database specifically. They also work together, though not as often.
- The final developer works on creating and uploading data to the database. They work together with the previous two developers to ensure full database connection.
- With each major change, each set of developers uploads their changed files to the main repository. At the end of each week, code reviews take place, and the branches are merged.

Performance Optimization Plan

There are a few potential bottlenecks for the LMS. This comes primarily from the large amount of media records a library keeps: some libraries can have thousands of books, among other types of media. The sheer number of records could slow down functions like finding a book or checking it out. However, the real issue might come with other types of media the LMS archives. If the LMS also contains audiovisual media types, storing thumbnail images or audio samples could present a huge burden on both storage and data transfer.

To deal with this, indexing will be used to make retrieval quicker. Everything should be stored in logical locations with IDs. Also, all data types with large file sizes will be compressed and encoded to make file transfers quicker. If possible, the LMS will also support cookie storage in user browsers to reduce the number of data transfers happening with each site visit.

Maintenance and Support Strategy

Post-deployment, maintenance will be handled by a portion of the total development team, specifically those trained in system and server maintenance. A database administrator will also be on-hand. Maintenance will be mostly corrective and perfective, to handle bugs and to improve performance, respectively. About once a month, the team will meet to discuss additional functions for the LMS. If agreed on, adaptive maintenance is performed to implement the new feature (support for a new media format, for example). Version control will be handled through the GitHub as previously mentioned, but the project manager will decide when to push file updates to the main system. All code files will be commented, and a single documentation file will be pushed with the system to describe its sum purpose and how to use it.

Finally, users will have multiple forms available to submit feedback to the developers. Feedback could range from bug reports to feature requests which would all be discussed in the weekly meeting.

Midterm System Implementation Plan

(Some sections are prior iterations of the above ones.)

Implementation in SDLC

System implementation is typically the fourth of five phases in the systems development life cycle. First, you must make a plan to see what problems need to be solved. Then, you have to make an analysis of the requirements of the new system and put together a logical model – just an outline of what the system will do. Then comes the design portion of the SDLC; a physical, working model of the system is created, and specification and documentation are delivered with it. Finally, there is implementation (with systems support after it). The system is fully constructed and implemented alongside the rest of the company's systems.

Implementation is what is being done here. The planning has been done, the analysis was completed, and the physical model of the Library Management System (LMS) has been constructed. Now, the system must be implemented into the work environment.

Chosen Development Methodology

The software development method chosen for this project is a general spiral methodology—iteration and user feedback are important. The LMS is meant to automate a lot of manual labor, but it's also important that users of the system can interact with it easily and efficiently. The graphical user interface, for instance, is a major part of the system that would need to be revised over time after surveying different demographics of users.

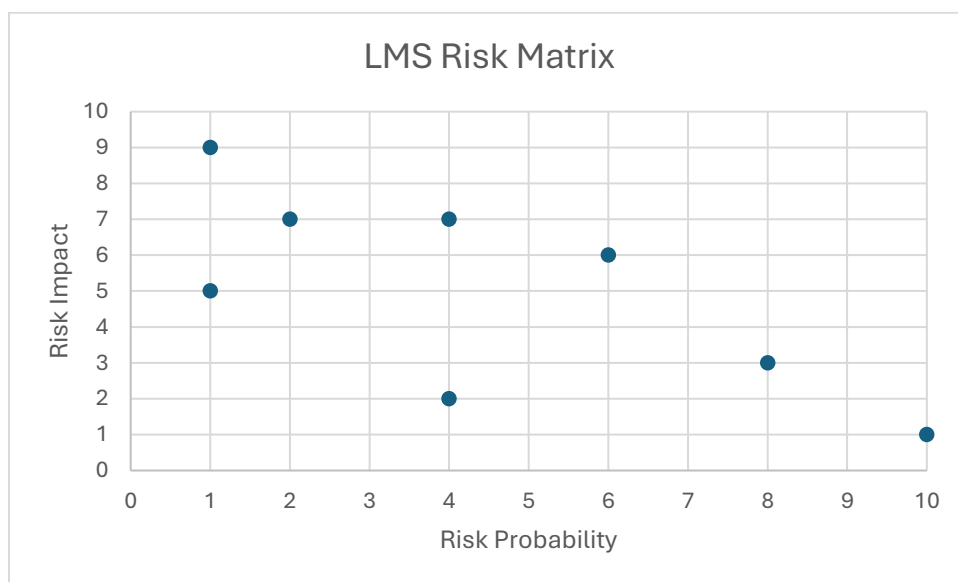
Planning Objectives and Strategies

There are several key goals I'd like to accomplish during system implementation planning. First, I'd like to create a tight, detailed schedule, but one that also has room for unplanned delays. A schedule that does not count on any step being delayed is a schedule that will not survive development and implementation. Second, I'd like to create a risk identification, assessment, and management plan to help better inform the schedule. Next, of course, is dealing with the various intricacies of system deployment: data migration, testing, debugging, iteration, etc. Finally, I'd like to make a plan for system support after implementation is complete.

Risk Assessment

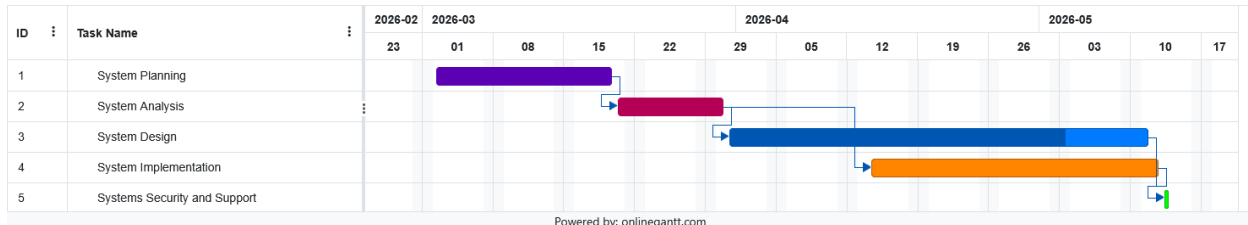
Risk is part of any development process, so it's important to account for anything that could come up, no matter how improbable. Thankfully, the Library Management System is far from complex and is not intrinsically urgent, so the probability and impact of certain risks are reduced compared to management systems in the healthcare or banking domains. Below are some examples of possible risks and a matrix for simple analysis. Risks are ranked on a scale from 1-10 with 1 being lowest in scale and 10 being highest in scale: a risk with an impact of 8 and a probability of 1 has a high impact, but a very low probability of occurring.

1. Development stalls due to extended worker issues. Since the team for creating the LMS is small, any worker shortage could cause issues. An artist quitting, or a programmer missing long periods of work could lead to delays. **Probability: 4, Impact: 7.**
2. In contrast to the previous issue, a single employee takes an extended break. They will undoubtedly return, but due to the small team, work might be slightly delayed. **Probability: 8, Impact: 3.**
3. Budget runs over due to lack of money. Libraries are often underfunded, so the costs of resources can become daunting. Running low on or out of money can be likely, unless extensive cost-cutting is implemented. **Probability: 6, Impact: 6.**
4. The physical server room is robbed or destroyed. The server room would be the only physical location (aside from the library itself) to worry about since development for the LMS would be done almost entirely remotely. Losing the servers itself would be a terrible loss of resources, but the room itself could be replaced. **Probability: 1, Impact: 5.**
5. Development tools become permanently unavailable (whether due to being taken down by the owners or licenses expiring). If open-source software is not used to create the LMS, this could prove to be disastrous during the design phase. Having to migrate tools could cause long delays. **Probability: 2, Impact: 7.**
6. Project files are temporarily lost, but backups remain in an offsite location. With good data storage practices, any single data loss will only be a short, temporary setback. **Probability: 4, Impact: 2.**
7. Project files and all backups are permanently lost. In this very unlikely scenario, design will have to begin again from scratch (with gained knowledge and experience). **Probability: 1, Impact: 9.**
8. Work on one aspect of the system stalls due to temporary roadblock. If a programmer finds a bug they cannot fix, or a hardware installer discovers there is no readily available power in the server room, work could very temporarily slow down. Any manner of obstruction could happen. **Probability: 10, Impact: 1.**



Project Timeline

The first four phases of the SDLC will take place over the course of two and a half working months. Implementation specifically will take approximately three weeks to complete. The Gantt chart below depicts the estimated progress of the project as the Systems Implementation phase is entered. The last phase—Systems Security and Support—has a duration of one day but will be continued indefinitely.



Installation Approach

For the system installation approach, I absolutely believe that a parallel operation changeover would be the best way to make the transition. As the library previously had no automated systems for management, both the library members and the library staff might have a difficult time transitioning from manual to automated. Directly cutting over would be too sudden a shift, and a pilot operation would not sufficiently stress test the new system (since we are using a spiral development approach). That leaves the phased and parallel operations as the two remaining installation choices. I believe parallel would be the superior choice because it would allow all users time to get used to the new system, and it would have a sufficient number of users for testing and feedback. Additionally, there aren't enough moving parts in the LMS to necessitate a phased operation changeover.

Installation Schedule

As mentioned before, installation of the LMS will take about three weeks. Below is an estimated schedule of activities that will have to be completed before installation is finished.

Days **1-14** – System is fully constructed based off of logical and physical models in design specifications.

Days **15-18** – Internal bug testing and QA are conducted, last minute changes are made.

Days **18-19** – Select server room in library and prepare room for hardware installation.

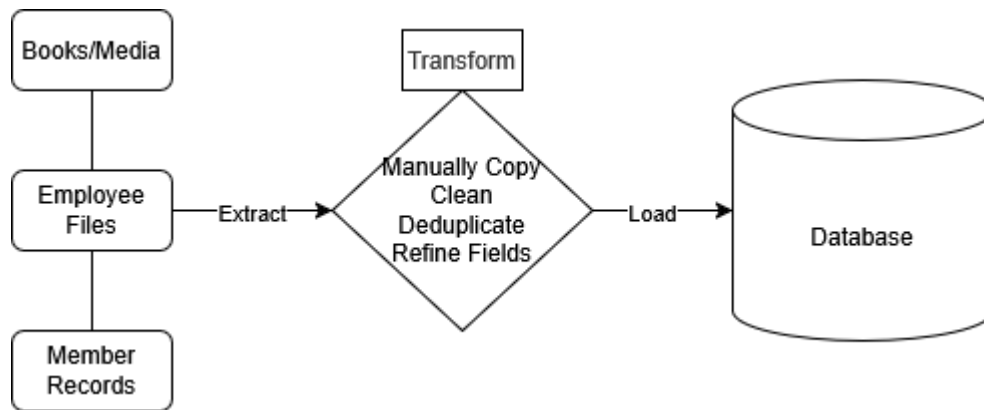
Day **20** – Install server architecture in server room.

Day **21** – Start server and begin access to the LMS.

Since a parallel operation approach is what's being used in this case, pen and paper records will continue to be used for a short while after installation so that the development team has time to collect feedback from users and modify the system to conform.

Data Migration Plan [ETL Diagram]

Once again, the fact that the previous system was pen, paper, and file proves to be a detriment for the new system implementation process. Rather than an easier transfer from digital to digital, data has to be manually copied from physical to digital. (Another possible solution would be to make photocopies/scans of all possible files and then use a machine learning algorithm to copy text and transfer to the database.) At least one person would have to copy all files into the new database, which could take quite a while depending on the size of the library. Data will have to be cleaned (due to human error), deduplicated (if there are duplicate records), and finally fields will have to be refined since physical databases can have some differences compared to digital ones.



Test Plan

Design a Test Plan outlining the scope, objectives, tools, and responsibilities.

Testing for the LMS will cover the usual four areas: unit, integration, system, and acceptance. Unit testing will be comprised of testing each individual function for the LMS to ensure all run without error given some test data (e.g., a login script, an “add book” function). Then, all connected functions and scripts will need to be tested together, likely by inserting test data into one function and then handing the results of that function to the next (like checking out a book for the logged in user and then changing that book to unavailable in the DB). System testing will occur when the previous two tests have finished: testers will run through a complete version of the site to ensure that everything works as specified. Finally, acceptance testing will occur using the userbase as testers: library staff and members. If feedback is given on changes, these will be analyzed and possibly incorporated into the next iteration of the LMS. Most tools used will be IDEs or virtual environments; if needed, automatic tools like postman will be used to check data flows and ensure server connections are stable and working.

Test Cases

Create two or more detailed test cases (e.g., login validation, transaction processing).

Test Case ID	Description	Precondition	Input	Expected Output
TC-F-011	Member attempts to	User account is member level	- Book: Animal Farm	Book checkout fails, “This book is [Checked Out],

	check out a book that is not available.	or higher and chosen book is not available.	[Checked Out]	please submit a hold request or choose another book!"
TC-V-004	Email does not comply with regex email check.	User has entered an incorrect email address into the email field.	- Email: lmsuser1.com	Account creation disabled, email field warning, "Email is invalid! Please check to make sure email is formatted correctly."

Debugging and Error Handling Strategies

Debugging must be done before the new system is deployed, but there is also room for user testing afterward. In other words, the system must be free of large, functional bugs before deployment. Different types of errors will need different strategies: server connection errors will require a review of the server architecture and functions that connect the client to the server to the database, functional errors will need a review of the application scripts, syntax errors typically point to themselves, etc. Using an IDE rather than a word processor makes error discovery and fixing much simpler.

Automated Testing Tools

As mentioned in the Test Plan, automatic tools will see limited usage, if any. The LMS is a conceptually and mechanically simple client-server application. Most testing and debugging can be done manually, saving on cost. Filling the database must also be handled manually, so it makes sense to clean and test the data manually too. Barring any unexpected sudden design changes, Postman would likely be the only automated tool to be used, for server connection testing purposes.

Data Quality Issues and Conversion Method

The main issue to grapple with here is the fact that data has to be transcribed from a physical format to a digital one. Whoever's filling in the database has to sit down and do the tedious job of reading from file row by row and then typing that into the DB, while also fixing capitalization, spelling errors, and duplication as they go (caused by human error when writing by file). Copying is also dependent on the handwriting of the files. What's more, filling the database by hand is also subject to human error, so there should be at least two people to check each other's work. Aside from the aforementioned tasks, general cleaning will work by filling in empty fields or removing records for which there isn't enough information to fill in.

Testing and Validation Steps

Validation scripts can be written to test the newly filled database for its accuracy and integrity. For example: testing emails against a regex to ensure that they're structurally sound addresses. A program could also be written to extract all records with empty fields (and/or fields with nonstandard data types) for manual overview. The tester can then fill these fields using an outside data source if necessary (e.g., a member address without a zip code, a book title without an author).

Summary of System Implementation Plan

In summary, system implementation must go through six main steps:

1. System software construction from logical and physical models.
2. Hardware construction.
3. Software and hardware connection.
4. Data cleaning and copying.
5. System and data testing and validation.
6. System deployment and user testing.

All of these steps serve some important purpose such that system installation cannot proceed without all of the mentioned steps being completed.

Key Success Factors and Post-Implementation Review Criteria

System installation success will be measured against the specified needs outlined in the system plan. If all of the needs have been met (e.g., a system where a user can check out books, a system where an administrator can accept a staff employment application), then the system will have succeeded. Of course, plans and needs are ever-changing; the library staff could decide that they want a new feature halfway through development, and whether to create that feature now or after deployment must be decided on. The system should also be free of all functional and validation-related bugs; minor bugs like display issues on less commonly used web-connected devices can be handled during user testing (especially since the parallel operation system installation approach is being utilized).

Lessons Learned

The main lesson learned is that system implementation is much more difficult than it seems, perhaps being the most difficult and complex phase in the Software Development Life Cycle. Like most things though, software development is a continuous process, and experience (even failure) is a useful guide through this process. The development team may learn from a failure in hardware installation to allot more time to that step in the future, as well as to vet their hardware engineers better. They may learn from a sudden change in the client's requirements to make sure that their needs are more explicitly understood. There's much to learn from on any project and adding lessons learned to the team's abilities helps future projects run smoother.